

Efficient Facial Expression Recognition using HOG with Python Optimization

Group 32

Xibo Xu (xx2316), Yating Xu (yx3710), Handing Ying (hy3104),
Juechen Zhong (jz4440), Tim Zhou (zz3645)

1 Introduction

Facial Expression Recognition (FER) is a fundamental task in computer vision, aiming to classify human emotions from facial images. It serves as a core component in applications such as human-computer interaction and affective computing. Traditional approaches rely on handcrafted features and classical models, such as extracting local edge and shape information using Histogram of Oriented Gradients (HOG) and performing classification with Logistic Regression.

However, HOG feature extraction involves intensive pixel-level computations and becomes a performance bottleneck when scaling to large datasets. Standard Python implementations or naive NumPy-based approaches often fail to achieve sufficient efficiency.

In this project, we focus on improving computational efficiency from a Python implementation perspective while keeping the underlying model unchanged. We investigate several optimization strategies, including NumPy vectorization, Just-In-Time (JIT) acceleration with Numba, and parallel computing. We further analyze their interactions, particularly the performance degradation caused by nested parallelism.

Our experiments show that NumPy vectorization achieves a $4\text{--}5\times$ speedup, while Numba provides over $200\times$ acceleration compared to the baseline. However, adding multiprocessing on top of an already fast Numba-based implementation can degrade performance because process-management and communication overhead can outweigh the remaining per-image computation.

2 Method

We study facial expression recognition on the FER2013 dataset, where each input image $x \in \mathbb{R}^{48 \times 48}$ is a grayscale face image and the goal is to predict an emotion label $y \in \{1, \dots, C\}$. Rather than training directly on raw pixels, we represent each image using Histogram of Oriented Gradients (HOG), which captures local edge structure and facial shape information. The overall pipeline is

$$x \rightarrow \mathbf{h}(x) \rightarrow f(\mathbf{h}(x)) \rightarrow y,$$

where $\mathbf{h}(x)$ is the HOG feature vector and f is a supervised linear classifier.

For each image, we first compute horizontal and vertical intensity gradients, denoted by G_x and G_y . From these gradients, the gradient magnitude and orientation at each pixel are computed as

$$m(i, j) = \sqrt{G_x(i, j)^2 + G_y(i, j)^2},$$

$$\theta(i, j) = \arctan\left(\frac{G_y(i, j)}{G_x(i, j)}\right)$$

Following the standard unsigned-gradient setting in HOG, orientations are mapped into the range $[0, 180^\circ)$. Intuitively, $m(i, j)$ measures how strong the local edge is, while $\theta(i, j)$ indicates its direction.

The image is then divided into small spatial cells. In our implementation, each FER2013 image has size 48×48 , and we use a cell size of 8×8 pixels, giving

$$\frac{48}{8} \times \frac{48}{8} = 6 \times 6$$

cells per image. For each cell, we build a histogram of gradient orientations using 9 bins over $[0, 180^\circ)$. Each pixel contributes a weighted vote to one orientation bin. The bin is determined by its orientation $\theta(i, j)$, and the vote weight is its gradient magnitude $m(i, j)$. Thus, if $\mathcal{C}$ denotes a cell and b denotes an orientation bin, the histogram entry can be written as

$$H_{\mathcal{C}}(b) = \sum_{(i,j) \in \mathcal{C}} m(i, j) \mathbf{1}[\theta(i, j) \in \text{bin } b],$$

To improve robustness to illumination and contrast changes, neighboring cells are grouped into overlapping blocks for normalization. We use a block size of 2×2 cells and a stride of 1 cell. Therefore, a 6×6 cell grid produces

$$(6 - 2 + 1) \times (6 - 2 + 1) = 5 \times 5 = 25$$

overlapping blocks. Each block contains $2 \times 2 \times 9 = 36$ histogram values, obtained by concatenating the four cell histograms. Let $\mathbf{v}$ denote the unnormalized block descriptor. We apply L_2 normalization:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\sqrt{\|\mathbf{v}\|_2^2 + \epsilon^2}},$$

The final HOG descriptor is formed by concatenating all normalized block descriptors:

$$\mathbf{h}(x) = [\hat{\mathbf{v}}_1, \hat{\mathbf{v}}_2, \dots, \hat{\mathbf{v}}_{25}].$$

Hence, the total feature dimension is

$$25 \times 36 = 900$$

The resulting 900-dimensional HOG vector is used as the input to a supervised classifier. In this work, we use logistic regression for classification.

3 Optimization

To improve the efficiency of HOG feature extraction, we implement three optimization strategies aligned with different components of the pipeline: gradient computation, histogram construction, and batch processing.

3.1 NumPy Vectorization

NumPy vectorization is applied to gradient computation and element-wise transformations. Specifically, image gradients are computed using array slicing operations (e.g., horizontal and vertical differences), avoiding

explicit Python loops. The gradient magnitude and orientation are computed using NumPy functions such as `np.hypot` and `np.arctan2`, followed by vectorized normalization of orientation values.

These operations are implemented entirely using array-based computations, which leverage optimized C-level routines and significantly reduce interpreter overhead for pixel-wise processing.

3.2 Numba Acceleration

Numba is used to accelerate the histogram construction and block normalization steps, which involve deeply nested loops over cells, pixels, and orientation bins. In particular, the accumulation of gradient magnitudes into orientation histograms and the subsequent block-wise normalization are implemented in a loop-based function compiled with `@njit`.

This allows the implementation to retain its explicit loop structure while achieving near C-level execution speed, which is especially effective for irregular memory access patterns and control-heavy computations that are not easily vectorized.

3.3 Parallel Processing

Parallel processing is applied at the dataset level by distributing HOG feature extraction across multiple images. A worker function is defined to compute features for a single image using a specified implementation (baseline, NumPy, or Numba), and multiple workers are executed concurrently using `ProcessPoolExecutor`.

Each image is processed independently, enabling efficient utilization of multiple CPU cores. The results are collected and assembled into a feature matrix, while progress is optionally tracked during execution.

4 Experiments

4.1 Setup

We evaluated the baseline build and four HOG implementation variants built from the three optimization strategies described above. The first variant uses NumPy alone to accelerate gradient computation and other array operations relative to the baseline loop-based implementation. The second retains NumPy for preprocessing and uses Numba to speed up the histogram construction and block normalization stages. The third adds multiprocessing on top of the NumPy version to enable multiple images to be processed concurrently across worker processes. The fourth integrates all three strategies into a single implementation.

4.2 Results

According to Table 1, the implementations can be ordered by training-set HOG throughput from lowest to highest as follows: baseline $\Rightarrow$ numpy $\Rightarrow$ numpy+parallel $\Rightarrow$ numpy+numba+parallel $\Rightarrow$ numpy+numba.

5 Discussion

The results show a clear improvement in HOG throughput from baseline to numpy, with further gains from numpy-parallel, numpy-numba-parallel, and the best performance achieved by numpy-numba. The speedup

Implementation	HOG on Training Set (images/s)	Test Accuracy (%)
baseline	64.8	45.04
numpy	277.7	45.04
numpy+numba	15272.9	45.17
numpy+parallel	1754.7	45.04
numpy+numba+parallel	5754.8	45.17

Table 1: Training-set HOG extraction speed and combined test accuracy for the five implementations using logistic regression.

from numpy comes from reducing Python-level overhead through vectorized array operations, while the much larger improvement in numpy-numba indicates that the loop-heavy stages of histogram accumulation and block normalization benefit strongly from JIT compilation. Although parallelism improved the numpy implementation, numpy-numba-parallel remained slower than numpy-numba, suggesting that once per-image computation became sufficiently cheap, multiprocessing overhead outweighed its benefit.

Additionally, in principle, these optimizations do not change the mathematical definition of HOG, so all five implementations are intended to produce equivalent features. However, small differences in floating-point rounding and accumulation order led to slight variation in the final classifier results.

6 Conclusion

In this project, we built a complete FER2013 emotion-recognition pipeline using a classical HOG-based representation and compared several optimization strategies for feature extraction. The experiments showed that numpy-numba provided the strongest speedup, while parallel processing was only helpful when the per-image workload remained sufficiently large. Importantly, these optimizations did not significantly change classification accuracy, which stayed near 45% across runs. This suggests that the main contribution of the project lies in improving runtime efficiency rather than predictive accuracy, and highlights the value of vectorization and JIT compilation in accelerating traditional data-science workflows.

Acknowledgements

The authors acknowledge the usage of OpenAI Codex (version 5.4), in the preparation of this project. Codex was employed in the following manners within this assignment: learning HOG’s algorithm, code development and debugging, and refining the phrasing and clarity of written explanations. All technical decisions, implementations, results, and interpretations were reviewed and verified by the authors, who take full responsibility for the final content.

We also thank Professor Majid Haghani Rizi and the teaching assistants of the *Advanced Python for Data Science* course at New York University for their discussions and feedback throughout the project.